

IF2211 Teori Bahasa Formal dan Automata

Milestone 1

Lexical Analysis

Laporan Tugas Besar

Disusun untuk memenuhi tugas mata kuliah IF2224 Teori Bahasa Formal dan Otomata
pada Semester V Tahun Akademik 2025/2026



Disusun Oleh

Dita Maheswari	13523125
Boye Mangaratua Ginting	13523127
Samantha Laqueenna Ginting	13523138
Lukas Raja Agripa	13523158

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

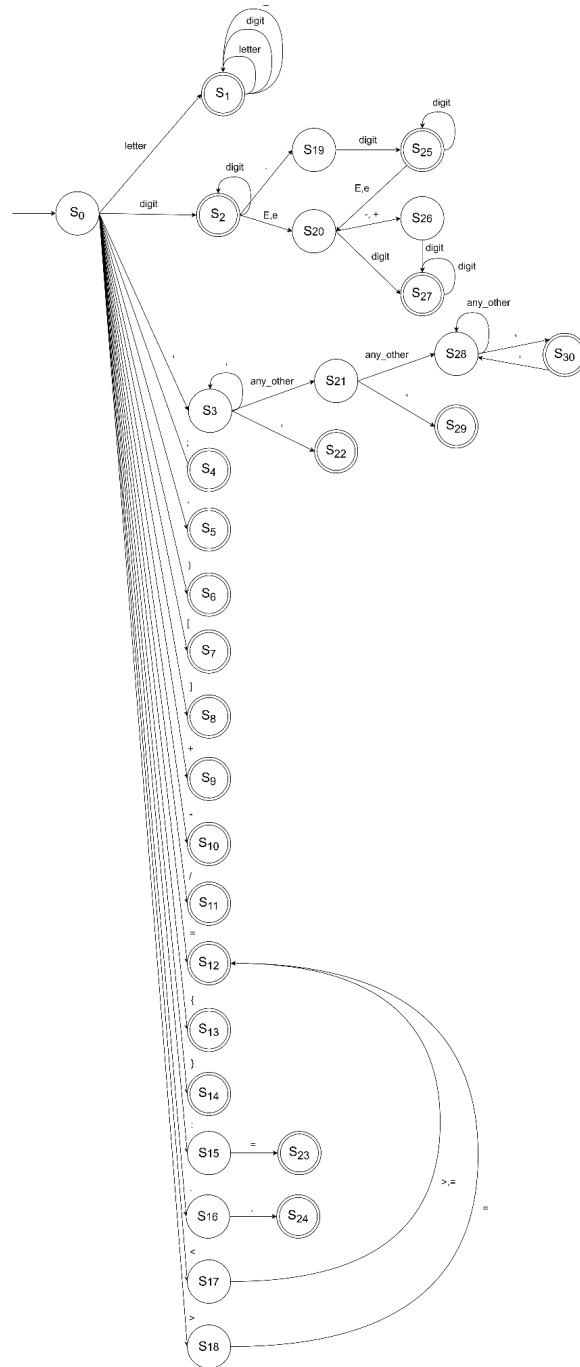
DAFTAR ISI	2
BAB 1 LANDASAN TEORI	3
BAB 2 PERANCANGAN DAN IMPLEMENTASI	4
2.1 Diagram	4
2.2 Aturan DFA	6
2.3 Implementasi Lexical Analyzer	8
BAB 3 PENGUJIAN	17
BAB 4 KESIMPULAN DAN SARAN	26
4.1 Kesimpulan	26
4.2 Saran	26
LAMPIRAN	27
A. Github	27
B. WorkSpace Diagram	27
C. Pembagian Tugas	27
REFERENSI	28

BAB 1 LANDASAN TEORI

Lexical analysis (analisis leksikal) merupakan sebuah proses pemecahan teks input menjadi token (*lexeme*), di mana token adalah unit dasar bermakna seperti *keyword*, *identifier*, *operator*, dan sebagainya. Pada tahap ini, karakter-karakter dari kode sumber dibaca secara berurutan dan dikelompokkan menjadi unit terkecil yang bermakna, yang disebut token. Token ini bisa berupa keyword (seperti *if*, *else*), identifier (nama variabel atau fungsi), konstanta (angka, string, boolean), operator (aritmatika, logika), delimiter (tanda baca seperti koma, titik koma, kurung), dan whitespace (spasi, enter) yang biasanya diabaikan.

Lexical analysis dikenal juga sebagai *scanning*. Proses ini memiliki fungsi utama memecah input menjadi lexeme (substring terkecil yang cocok dengan pola token) dan mengkategorikannya menjadi token yang mewakili elemen-elemen sintaksis seperti *keyword*, *identifier*, *operator*, dan *literal*. Metode yang umum digunakan dalam *lexical analysis* meliputi penggunaan *regular expressions* dan *finite automata* untuk mengenali pola token. Algoritma *lexical analysis* biasanya menggunakan metode ini untuk mengenali dan memvalidasi token serta menghilangkan karakter-karakter yang tidak diperlukan seperti komentar dan spasi kosong.

BAB 2 PERANCANGAN DAN IMPLEMENTASI



2.1 Diagram

Dengan Keterangan State sebagai berikut:

```

S0 : START (Bukan final)
S1 : IDENTIFIER / KEYWORD (FINAL)
S2 : NUMBER (INTEGER) (FINAL)
S3 : QUOTE_OPEN (') (Bukan final) - masuk ke literal state
S4 : SEMICOLON ; (FINAL)
S5 : RPAREN ) (FINAL)
S6 : LPAREN ( (FINAL)
S7 : LBRACKET [ (FINAL)
S8 : RBRACKET ] (FINAL)
S9 : ARITHMETIC_OPERATOR + (FINAL)
S10 : ARITHMETIC_OPERATOR - (FINAL)
S11 : ARITHMETIC_OPERATOR / (FINAL)
S12 : ASSIGNABLE/EQUAL START = (FINAL as =)
S13 : LBRACE { (FINAL)
S14 : RBRACE } (FINAL)
S15 : COLON : (Bukan final) - bisa jadi : final (COLON) atau jika
diikuti =, ASSIGN_OPERATOR := (S23)
S16 : DOT . (Bukan final) - bisa jadi single DOT (DOT) atau PART OF
RANGE .. (S24)
S17 : LESS_THAN < (Bukan final) - could be < (RELATIONAL) or <= or
<> depending next char
S18 : GREATER_THAN > (Bukan final) - could be > or >=
S19 : DOT_SEEN_WITHOUT_LEADING_DIGIT (Bukan final) - transisi
khusus bila . datang dari S0 (dibedakan dari decimal point setelah
digits). (Bergantung implementasi bisa diperlakukan sebagai DOT
final)
S20 : Decimal_Point(Bukan final) - state setelah melihat . dari S2
(digit . ), menunggu decimal
S21 : IN_STRING_LOOP (Bukan final) - menerima any_other chars
inside '...' (loop)
S22 : CHAR_LITERAL (FINAL) - terlihat sebagai path kalau 'X'
(single char) dan langsung close → finalize CHAR_LITERAL
S23 : ASSIGN_OPERATOR := or RELATIONAL <= or == (FINAL) - state
penampung untuk kombinasi := → := OR = after < → <= (grafik
menunjukkan panah ke S23 dari S15 dan S17)
S24 : RANGE_OPERATOR .. or RELATIONAL >= (FINAL) - state untuk .+.
→ .. atau >+ → >= (gambar menunjukkan S16→S24 dan garis besar ke
S24)
S25 : Decimal_digits(FINAL) - state loop digit setelah decimal
point (hasil → NUMBER (FLOAT))
S26 : EXP_MARK E/e (Bukan final) - state setelah melihat E/e in
float, menunggu optional sign or digits
S27 : EXP_DIGITS (FINAL) - loop digit for exponent (hasil → NUMBER
(scientific))
S28 : POSSIBLE_STRING_CLOSE_CHECK (Bukan final) - state setelah
reading closing '?' (digambar ada loop any_other di S28)
S29 : UNKNOWN (FINAL atau Bukan tergantung implementasi) - used for
single other tokens or error sink
S30 : STRING_LITERAL (FINAL) - final state untuk string ('...' with
length > 1)

```

2.2 Aturan DFA

Berikut merupakan penjelasan aturan DFA yang telah dibuat:

1. State START

- State ini merupakan titik awal DFA. Berdasarkan karakter pertama dari input, sistem akan berpindah ke state lain untuk menentukan jenis token yang sesuai:
- Jika karakter pertama berupa huruf → menuju IN_IDENTIFIER.
- Jika berupa angka → menuju IN_INTEGER.
- Jika berupa tanda kutip tunggal ' → menuju IN_QUOTE untuk memproses character atau string literal.
- Jika berupa simbol tertentu seperti ;, +, -, /, =, (,), [,], dan sebagainya → langsung menuju state final yang sesuai, seperti SEMICOLON_FINAL, ARITHMETIC_OPERATOR_FINAL, dan lainnya.
- Jika berupa . → menuju DOT_STATE (kemungkinan operator range ..).
- Jika berupa : → menuju COLON_STATE (kemungkinan operator :=).
- Jika berupa < atau > → menuju LT_STATE atau GT_STATE (kemungkinan <=, >=, <>).
- Jika berupa (→ menuju LPAREN_STATE (kemungkinan awal komentar (*).
- Jika berupa * → menuju STAR_STATE (kemungkinan akhir komentar *)).

2. Identifier (IN_IDENTIFIER)

- Merupakan accepting (final) state dengan token bertipe IDENTIFIER.
- Menerima kombinasi huruf, angka, dan underscore (_), dengan transisi ke dirinya sendiri.
- Setelah token terbentuk, DFA akan memeriksa apakah nilai lexeme termasuk dalam reserved words (kata kunci). Jika cocok, token diubah menjadi KEYWORD.

3. Number (Integer, Real, dan Eksponen)

State ini menangani berbagai jenis bilangan:

- IN_INTEGER → final state bertipe NUMBER. Jika ditemukan . maka berpindah ke AFTER_DOT, dan jika ditemukan E/e maka berpindah ke EXP_START.
- AFTER_DOT → menunggu digit berikutnya untuk berpindah ke IN_REAL.
- IN_REAL → final state bertipe NUMBER. Dapat berpindah ke EXP_START untuk notasi eksponen.
- EXP_START → menerima tanda + atau - (menuju EXP_SIGN) atau langsung digit (menuju EXP_DIGITS).
- EXP_SIGN → harus diikuti digit untuk menuju EXP_DIGITS.
- EXP_DIGITS → final state bertipe NUMBER dan menerima tambahan digit.

4. String dan Character Literal

Penanganan literal karakter dan string mengikuti gaya bahasa Pascal (menggunakan tanda kutip tunggal):

- `IN_QUOTE` → jika langsung diikuti `'`, maka menjadi `EMPTY_STRING_FINAL` (string kosong `""`). Jika karakter berikutnya bukan `'`, maka menuju `FIRST_CHAR`.
- `FIRST_CHAR` → jika setelah satu karakter muncul `'`, maka berpindah ke `POSSIBLE_CHAR_END` dan menghasilkan token `CHAR_LITERAL`. Jika bukan `'`, berpindah ke `IN_STRING_MULTI`.
- `IN_STRING_MULTI` → membaca karakter berulang hingga menemukan `'`, lalu menuju `POSSIBLE_STRING_END`.
- `POSSIBLE_STRING_END` → final state bertipe `STRING_LITERAL`. Jika ditemukan `'` lagi, kembali ke `IN_STRING_MULTI` (mendukung escape quote dengan menggandakan `'`, misalnya `don't` → menghasilkan isi string `don't`).

Hasil akhir token dapat berupa:

- `CHAR_LITERAL` untuk satu karakter, seperti `'A'`
- `STRING_LITERAL` untuk string kosong atau panjang, seperti `'program'` atau `''`

5. Operators dan Delimiters

State ini menangani simbol-simbol operator dan tanda baca:

- Simbol tunggal:
 - `SEMICOLON_FINAL` → `;`
 - `COMMA_FINAL` → `,`
 - `RPAREN_FINAL`, `LBRACKET_FINAL`, `RBRACKET_FINAL`, dan lainnya sesuai simbolnya.
- Operator aritmatika:
 - `ARITHMETIC_OPERATOR_FINAL` menangani `+`, `-`, `*`, `/`
 - (Tanda `*` juga dapat menjadi bagian dari akhir komentar `*)` bila diikuti `)`).
- Operator relasional:
 - `RELATIONAL_OPERATOR_FINAL` menangani `=`, `<=`, `>=`, `<>`
- Operator penugasan:
 - `ASSIGN_OPERATOR_FINAL` menangani `:=`
- Operator range:
 - `RANGE_OPERATOR_FINAL` menangani `..`

6. Reserved Words

Beberapa identifier dikenali sebagai reserved word dan akan diubah menjadi jenis token tertentu.

- a. Keyword

Terdiri dari program, var, const, type, function, procedure, begin, end, if, then, else, while, do, for, to, downto, integer, real, boolean, char, array, of, true, false

b. Operator

Dibagi menjadi dua, yaitu:

- ARITHMETIC_OPERATOR → div, mod
- LOGICAL_OPERATOR → and, or, not

7. Penanganan Kasus Khusus dan Ambiguitas

Beberapa kasus ambiguitas ditangani secara deterministik melalui transisi khusus:

a. Range .. vs . Desimal

- Jika input dimulai dengan . (misal .123), maka dari START, DFA akan berpindah ke DOT_STATE
Jika karakter berikutnya juga . , maka menjadi RANGE_OPERATOR_FINAL
- Untuk angka desimal seperti 0.5, input dimulai dari digit sehingga berpindah ke IN_INTEGER, lalu ke AFTER_DOT, dan akhirnya IN_REAL

b. Operator Penugasan : vs :=

- COLON_STATE merupakan final state bertipe COLON. Jika diikuti dengan = , maka berpindah ke ASSIGN_OPERATOR_FINAL (token :=)

c. Operator Relasional <, >, <=, >=, <>

- LT_STATE adalah final state bertipe RELATIONAL_OPERATOR. Jika diikuti = atau > , maka berpindah ke RELATIONAL_OPERATOR_FINAL (<=, <>)
- GT_STATE jika diikuti = , maka menghasilkan token >=

d. String Escaping

- Dua tanda kutip tunggal (') di dalam string diartikan sebagai satu tanda kutip (') di dalam nilai string. Hal ini diimplementasikan melalui transisi POSSIBLE_STRING_END → ' → IN_STRING_MULTI

2.3 Implementasi Lexical Analyzer

Untuk tahap Lexical Analyzer, kami menggunakan bahasa pemrograman *Python* berbasis objek, dengan struktur direktori berikut :

a. Model

- DFA.py
- Token.py
- Lexer.py
- State.py

- b. Repository
 - DFAParser.py
 - JSONParser.py
 - Parser.py
 - TokenType.py
- c. View
 - Services.py
- d. main.py
- e. run.py

2.3.1 Class DFAParser

2.3.1.1. Atribut DFAParser

Atribut	Penjelasan
Tidak ada Atribut	

2.3.1.2 Method DFAParser

Method	Penjelasan
<code>def parseDFAFromJSON(self, filePath: str) -> DFA:</code>	Func keseluruhan untuk mendapatkan object DFA dengan parsing JSON dfa.json
<code>def buildDFA(self, dfaData: Dict[str, Any]) -> DFA:</code>	Func untuk membuat objek DFA dari data JSON

2.3.2 Class JSONParser

2.3.1.1. Atribut JSONParser

Atribut	Penjelasan
Tidak ada Atribut	

2.3.1.2 Method JSONParser

Method	Penjelasan
<code>def parseFile(self, filePath: str) -> str:</code>	Func inti untuk melakukan baca file, untuk ini buat JSON
<code>def parseJSON(self, jsonString: str) -> Dict[str, Any]:</code>	Func untuk melakukan parseJSON setelah baca JSON dari func diatas.

2.3.3 Class Parser (extends JSONParser)

2.3.1.1. Atribut Parser

Atribut	Penjelasan
Tidak ada Atribut	

2.3.1.2 Method DFAState

Method	Penjelasan
<pre>def parseDFA(self, filePath: str) -> DFA:</pre>	Func enkapsulasi dari func parser inti sebelumnya, yang mana ini dipanggil oleh loadDFA dalam Services/Antarmuka user. Tentu saja, terdapat pengecekan, harus file json.

2.3.3 Class TokenType

2.3.1.1. Atribut TokenType

Atribut	Penjelasan
Berisikan object dengan fungsi enum auto() supaya menghasilkan nilai pada Tiap Token yang didefinisikan. Karena dalam hal ini, tidak penting untuk mendapatkan nilai numerik dari tiap tokennya. <pre>KEYWORD = auto() # begin, end, if, else, then, while, do, var, const, procedure, function, return, true, false, null IDENTIFIER = auto() # nama variabel, nama fungsi, nama prosedur, STRING_LITERAL = auto() # string dalam tanda kutip ganda CHAR_LITERAL = auto() # karakter dalam tanda kutip tunggal NUMBER_LITERAL = auto() # angka BOOLEAN_LITERAL = auto() # true atau false NULL_LITERAL = auto() # null OPERATOR = auto() # +, -, *, /, =, <, >, <=, >=, ==, != DELIMITER = auto() # ; , () { } [] :</pre> Dan seterusnya....	

2.3.1.2 Method TokenType

Method	Penjelasan
Tidak ada Method	

2.3.3 Class State

2.3.1.1. Atribut State

Atribut	Penjelasan
<code>self.name</code>	Nama state
<code>self.isFinal</code>	Apakah state tersebut sudah final ?
<code>self.TypeToken</code>	Menyimpan tipe token
<code>self.transitions: Dict[str, 'State'] = {}</code>	Menyimpan Transisi dari suatu input, ke State berikutnya.

2.3.1.2 Method State

Method	Penjelasan
<code>def addTransition(self, inputChar: str, nextState: 'State'):</code>	Func untuk menambahkan transisi ketika mendapatkan suatu input
<code>def getNextState(self, inputChar: str) -> Optional['State']:</code>	Func getter State selanjutnya

2.3.3 Class Token

2.3.1.1. Atribut Token

Atribut	Penjelasan
<code>self.tokenType</code>	Tipe Token
<code>Self.value</code>	Nilai dari token
<code>self.line</code>	Posisi line
<code>self.column</code>	Posisi column

2.3.1.2 Method Token

Method	Penjelasan
<code>def __str__(self) -> str:</code>	Func untuk mengeluarkan output di terminal sesuai format di spek
<code>def __repr__(self) -> str:</code>	Func untuk memanggil <code>__str()</code>
<code>def getType(self) -> TokenType:</code>	Func getter tipetoken
<code>def getValue(self) -> str:</code>	Func getter nilai token

<code>def getPosition(self) -> tuple:</code>	Func getter posisi
---	--------------------

2.3.3 Class Lexer

2.3.1.1. Atribut Lexer

Atribut	Penjelasan
<code>self.dfa = dfa</code>	Objek DFA yang dibangun dan dinisialisasi untuk Lexical Analyzer
<code>Self.sourcode</code>	Isi kode program pascal.pas
<code>Self.position</code>	Menyimpan posisi line dan column
<code>self.linenummer</code>	Menyimpan posisi line untuk scanning
<code>self.columnnumber</code>	Menyimpan posisi column untuk scanning
<code>self.tokens</code>	Kumpulan token yang didapat

2.3.1.2 Method Lexer

Method	Penjelasan
<code>def setSourceCode(self, sourceCode: str):</code>	Func setter source code pascal sebelum scanning
<code>def getCurrentChar(self) -> Optional[str]:</code>	Func getter karakter sekarang
<code>def peekNextChar(self) -> Optional[str]:</code>	Func liat karakter selanjutnya, tapi ngga adv()
<code>def advance(self)</code>	Func adv(), seperti Data Struktur Mesin Kata dan Mesin Karakter – Untuk lompat ke karakter selanjutnya
<code>def isNegativeNumber(self) -> bool</code>	Func mengecek apakah termasuk bilangan Negatif, karena <i>asumsinya masih tahap awal yaitu Lexical Analyzer, yang masih menganggap bilangan negatif, adalah NUMBER(-), bukan OPERATOR(-)</i>
<code>def skipWhitespace(self): # Melewati whitespace (spasi, tab, newline, dll)</code>	Func skip Whitespace seperti tab, spaces, enter \t \n dsb
<code>def scanNumber(self, includeSign:</code>	Func untuk scan bilangan, dengan atau tanpa

<code>bool = False) -> Token:</code>	negatif
<code>def scanIdentifierOrKeyword(self) -> Token:</code>	Func untuk scanning dan menentukan apakah finalState harus menghasilkan Identifier atau Keyword dari berbagai input. Hal ini dibantu dengan func sebelumnya dengan Data Struktur Keyword = Set()
<code>def scanStringLiteral(self) -> Token:</code>	Func scan string literal , terpisah char literal
<code>def scanCharLiteral(self) -> Token:</code>	Func scan char literal, terpisah dengan string literal
<code>def scanComment(self) -> Optional[Token]: Return None</code>	Func scan comment yang terdeteksi, dan tidak akan ditokenisasi.
<code>def scanOperator(self) -> Token</code>	Func scan Operator
<code>def tokenize(self) -> List[Token]:</code>	Func MAIN METHOD dari keseluruhan Func lexical Analyzer, sehingga dapat membaca kode program dan mereturn List of Token yang didapat
<code>def getTokens(self) -> List[Token]:</code>	Func getter List of Tokens

2.3.3 Class DFA

2.3.1.1. Atribut DFA

Atribut	Penjelasan
<code>self.states: Dict[str, State] = {}</code>	Kumpulan Objek State
<code>self.startState: Optional[State] = None</code>	Start State
<code>self.finalStates: Set[State] = set()</code>	Set of Objek State yang merupakan Final State
<code>self.keywords = set()</code>	Set of Keyword (agar memisahkan dari Token lainnya, seperti IDENTIFIER)

2.3.1.2 Method DFA

Method	Penjelasan
--------	------------

<pre>def addState(self, name: str, isFinal: bool = False, tokenType: Optional[TokenType] = None) -> State:</pre>	Func membuat State baru untuk keperluan Algoritma dalam DFA
<pre>def setStartState(self, stateName: str): #</pre>	Func setter start state
<pre>def addTransition(self, fromState: str, inputChar: str, toState: str):</pre>	Func tambahkan transisi antar state untuk keperluan Algoritma dalam DFA
<pre>def getCurrentState(self) -> State</pre>	Func getter state sekarang
<pre>def isKeyword(self, lexeme: str) -> bool:</pre>	Func apakah dia KEYWORD atau Token lain
<pre>def getTokenType(self, lexeme: str, finalState: State) -> TokenType:</pre>	Func getter TokenType untuk keperluan Algoritma dalam DFA

2.3.3 Class Services

2.3.1.1. Atribut Services

Atribut	Penjelasan
<code>self.parser</code>	Objek Parser untuk antarmuka User
<code>Self.lexer</code>	Objek Lexer untuk antarmuka User
<code>self.dfa</code>	Objek DFA untuk antarmuka User

2.3.1.2 Method Services

Method	Penjelasan
<pre>def loadDFA(self, dfaFilePath: str) -> DFA:</pre>	Func keseluruhan dari parsing File, parsing JSON untuk antarmuka User
<pre>def initializeLexer(self, dfaFilePath: str):</pre>	Func untuk inisialisasi Lexer
<pre>def loadPascalFile(self, filePath: str) -> str:</pre>	Func loader pascal file
<pre>def validatePascalFilePath(self, filePath: str) -> bool</pre>	Func validasi Pascal File path

<pre>def performLexicalAnalysis(self, sourceCode: str) -> List[Token]:</pre>	Func driver untuk Lexical Analysis
<pre>def displayTokens(self, tokens: List[Token]):</pre>	Func menampilkan tokenization
<pre>def processFile(self, pascalFilePath: str, dfaFilePath: str) -> List[Token]:</pre>	Func keseluruhan Services / Antarmuka user hasil enkapsulasi func lainnya (termasuk func diclass lain) untuk melakukan pemrosesan total. Ditaruh dalam main.py
<pre>def showErrorMessage(self, message: str = "err"): # Menampilkan pesan error print(f"ERROR: {message}")</pre>	Func untuk menampilkan pesan error (Exception)

2.3.3 Main & Run (Driver)

2.3.1.1. Atribut Services

Atribut	Penjelasan
<code>def main()</code>	Func menampung enkapsulasi func / entry point program utama ketika dijalankan
<code>run.py</code>	driver

2.3.x Alur Program

Program yang dibangun mengambil referensi dari Data Struktur Mesin Kata dan Mesin Karakter beserta OOP. Awalnya ketika program dijalankan melalui Main & Run sebagai driver, maka akan memanggil Func Main Method di Class Services, sebagai antarmuka user untuk menginput kode .pas (path global), kemudian akan divalidasikan dengan func di Services. Hal ini juga sekaligus akan melakukan Load pada JSON, dan melakukan parsing JSON dan dibentuknyalah objek DFA itu sendiri.

Objek DFA yang dibangun mengandung atribut seperti kumpulan objek State yang telah diconstruct, Transitions dan Set of Keywords, sebagaimana yang telah diparsing dari JSON. Objek DFA berperan dalam constructor dari Lexer (Lexical Analyzer sebagaimana merupakan kumpulan algoritma inti dari pengujian ini). Lexer yang telah diconstruct/dibangun akan melakukan tokenize() yang sebagai MAIN METHOD, dengan melakukan looping hingga mencapai EOF (end of file). Dan melakukan scanning (diambil dari func yang ada di Lexer dan telah dienkapsulasi disini). Yang membedakan dengan teori dari *youtube sebagai referensi di link [berikut](#)*, yang mengatakan bahwa objek Lexer akan melakukan penghilangan comment dan

whitespaces dahulu, namun pada algoritma disini sedikit berbeda, yaitu dengan looping dan scanning setiap karakter dengan tiap func scanning yang ada. Namun apabila ditemukan Whitespaces termasuk Comment, maka untuk Whitespaces akan diskip, dan untuk comment akan Return None.

Setelah EOF, maka akan mendapatkan List of Token dan akan dikembalikan ke Services, untuk ditampilkan kedalam terminal dengan format yang sudah dispesifikasikan dalam Milestone-1

BAB 3 PENGUJIAN

Berikut adalah pengujian yang telah dilakukan terdiri dari File .pas Input dan Output yang dihasilkan, *hal ini dapat terlihat dalam folder test/milestone1*

1.1. Program1.pas (input)

```
program helloworld;  
  
var  
    a, b: integer;  
  
begin  
    a := -0.123E;  
    b := a + 10;  
    writeln('Result = ', b);  
end.
```

1.2. Program1.txt (output)

```
KEYWORD(program)  
IDENTIFIER(helloworld)  
SEMICOLON(;  
KEYWORD(var)  
IDENTIFIER(a)  
COMMA(,  
IDENTIFIER(b)  
COLON(:)  
KEYWORD(integer)  
SEMICOLON(;  
KEYWORD(begin)  
IDENTIFIER(a)  
ASSIGN_OPERATOR(:=)  
NUMBER(-0.123E)  
SEMICOLON(;  
IDENTIFIER(b)  
ASSIGN_OPERATOR(:=)  
IDENTIFIER(a)  
ARITHMETIC_OPERATOR(+)  
NUMBER(10)  
SEMICOLON(;  
IDENTIFIER(writeln)  
LPARENTHESIS(  
STRING_LITERAL('Result = '  
COMMA(,  
IDENTIFIER(b)  
RPARENTHESIS())  
SEMICOLON(;
```

```
KEYWORD(end)
DOT(.)
```

2.1. Program2.pas (input)

```
program logika;
var
  a, b, c: integer;
  ok: boolean;
begin
  a := 10;
  b := 3;
  c := a div b + a mod b - (a - b) * 2;
  ok := (a >= b) and (b <> 0) or not (a < 0);
  if ok then
    c := c + 1
  else
    c := c - 1;
end.
```

2.2. Program2.txt (output)

```
KEYWORD(program)
IDENTIFIER(logika)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(a)
COMMA(,)
IDENTIFIER(b)
COMMA(,)
IDENTIFIER(c)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(ok)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(10)
SEMICOLON(;)
IDENTIFIER(b)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
IDENTIFIER(c)
```

```

ASSIGN_OPERATOR (:=)
IDENTIFIER (a)
KEYWORD (div)
IDENTIFIER (b)
ARITHMETIC_OPERATOR (+)
IDENTIFIER (a)
KEYWORD (mod)
IDENTIFIER (b)
ARITHMETIC_OPERATOR (-)
LPARENTHESIS ( ( )
IDENTIFIER (a)
ARITHMETIC_OPERATOR (-)
IDENTIFIER (b)
RPARENTHESIS ( ) )
ARITHMETIC_OPERATOR (*)
NUMBER (2)
SEMICOLON (;)
IDENTIFIER (ok)
ASSIGN_OPERATOR (:=)
LPARENTHESIS ( ( )
IDENTIFIER (a)
RELATIONAL_OPERATOR (>=)
IDENTIFIER (b)
RPARENTHESIS ( ) )
KEYWORD (and)
LPARENTHESIS ( ( )
IDENTIFIER (b)
RELATIONAL_OPERATOR (<>)
NUMBER (0)
RPARENTHESIS ( ) )
KEYWORD (or)
KEYWORD (not)
LPARENTHESIS ( ( )
IDENTIFIER (a)
RELATIONAL_OPERATOR (<)
NUMBER (0)
RPARENTHESIS ( ) )
SEMICOLON (;)
KEYWORD (if)
IDENTIFIER (ok)
KEYWORD (then)
IDENTIFIER (c)
ASSIGN_OPERATOR (:=)
IDENTIFIER (c)
ARITHMETIC_OPERATOR (+)
NUMBER (1)
KEYWORD (else)
IDENTIFIER (c)
ASSIGN_OPERATOR (:=)
IDENTIFIER (c)
ARITHMETIC_OPERATOR (-)

```

```
NUMBER(1)
SEMICOLON(;)
KEYWORD(end)
DOT(.)
```

3.1. Program3.pas (input)

```
program arrays;
var
  i: integer;
  arr: array[1..5] of integer;
begin
  for i := 1 to 5 do
    arr[i] := i * i;

  for i := 5 downto 1 do
    arr[i] := arr[i] + 1;

  if arr[1] <= arr[2] then
    arr[3] := arr[1] + arr[2];
end.
```

3.2. Program3.txt (output)

```
KEYWORD(program)
IDENTIFIER(arrays)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(i)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(arr)
COLON(:)
KEYWORD(array)
LBRACKET([)
NUMBER(1)
RANGE_OPERATOR(..)
NUMBER(5)
RBRACKET(])
KEYWORD(of)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
```

```

KEYWORD(to)
NUMBER(5)
KEYWORD(do)
IDENTIFIER(arr)
LBRACKET([)
IDENTIFIER(i)
RBRACKET(])
ASSIGN_OPERATOR(:=)
IDENTIFIER(i)
ARITHMETIC_OPERATOR(*)
IDENTIFIER(i)
SEMICOLON(;)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(5)
KEYWORD(downto)
NUMBER(1)
KEYWORD(do)
IDENTIFIER(arr)
LBRACKET([)
IDENTIFIER(i)
RBRACKET(])
ASSIGN_OPERATOR(:=)
IDENTIFIER(arr)
LBRACKET([)
IDENTIFIER(i)
RBRACKET(])
ARITHMETIC_OPERATOR(+)
NUMBER(1)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(arr)
LBRACKET([)
NUMBER(1)
RBRACKET(])
RELATIONAL_OPERATOR(<=)
IDENTIFIER(arr)
LBRACKET([)
NUMBER(2)
RBRACKET(])
KEYWORD(then)
IDENTIFIER(arr)
LBRACKET([)
NUMBER(3)
RBRACKET(])
ASSIGN_OPERATOR(:=)
IDENTIFIER(arr)
LBRACKET([)
NUMBER(1)
RBRACKET(])

```

```
ARITHMETIC_OPERATOR(+)  
IDENTIFIER(arr)  
LBRACKET([)  
NUMBER(2)  
RBRACKET(])  
SEMICOLON(;  
KEYWORD(end)  
DOT(.
```

4.1. Program4.pas (input)

```
program yangpentingdesimaldah;  
  
var  
    iniDesimal := 0.1213123  
    iniScientific := 0.123E22  
    samaNeg := -0.124E-10  
    biasa := 123.456  
  
begin  
    writeln(0.1213123)  
    writeln(0.123E22)  
    writeln(-0.124E-10)  
    writeln(123.456)  
    writeln("Dah pIndah")  
    writeln("Dah pIndah")  
    writeln("Dah pIndah")  
    writeln("Dah pIndah")  
    writeln("Dah pIndah")  
    writeln(iniScientific - samaNeg)  
    writeln(-0.124E-10 - (-0.124E-10))  
    writeln(biasa div 2)  
end.
```

4.2. Program4.txt (output)

```
KEYWORD(program)  
IDENTIFIER(yangpentingdesimaldah)  
SEMICOLON(;  
KEYWORD(var)  
IDENTIFIER(iniDesimal)  
ASSIGN_OPERATOR(:=)  
NUMBER(0.1213123)  
IDENTIFIER(iniScientific)  
ASSIGN_OPERATOR(:=)  
NUMBER(0.123E22)  
IDENTIFIER(samaNeg)  
ASSIGN_OPERATOR(:=)
```

```

NUMBER(-0.124E-10)
IDENTIFIER(biasa)
ASSIGN_OPERATOR(:=)
NUMBER(123.456)
KEYWORD(begin)
IDENTIFIER(writeln)
LPARENTHESIS(())
NUMBER(0.1213123)
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
NUMBER(0.123E22)
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
NUMBER(-0.124E-10)
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
NUMBER(123.456)
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
STRING_LITERAL("Dah pIndah")
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
STRING_LITERAL("Dah pIndah")
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
STRING_LITERAL("Dah pIndah")
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
STRING_LITERAL("Dah pIndah")
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
IDENTIFIER(iniScientific)
ARITHMETIC_OPERATOR(-)
IDENTIFIER(samaNeg)
RPARENTHESIS())
IDENTIFIER(writeln)
LPARENTHESIS(())
NUMBER(-0.124E-10)
ARITHMETIC_OPERATOR(-)

```

```

LPARENTHESIS ( )
NUMBER (-0.124E-10)
RPARENTHESIS ( )
RPARENTHESIS ( )
IDENTIFIER (writeln)
LPARENTHESIS ( )
IDENTIFIER (biasa)
KEYWORD (div)
NUMBER (2)
RPARENTHESIS ( )
KEYWORD (end)
DOT (.)

```

5.1. Program5.pas (input)

```

program COMMENT;
var
  ch: char;
begin
  { komentar gaya kurung kurawal }
  (* komentar gaya bintang-kurung *)
  ch := 'A';
  writeln('Dont panic: value = ', ch);
end.

```

5.2. Program5.txt (output)

```

KEYWORD (program)
IDENTIFIER (COMMENT)
SEMICOLON (;)
KEYWORD (var)
IDENTIFIER (ch)
COLON (:)
KEYWORD (char)
SEMICOLON (;)
KEYWORD (begin)
IDENTIFIER (ch)
ASSIGN_OPERATOR (:=)
CHAR_LITERAL ('A')
SEMICOLON (;)
IDENTIFIER (writeln)
LPARENTHESIS ( )
STRING_LITERAL ('Dont panic: value = ')
COMMA (,)
IDENTIFIER (ch)
RPARENTHESIS ( )
SEMICOLON (;)
KEYWORD (end)

```


DOT (.)

BAB 4 KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada milestone 1 tugas besar IF2224 ini berfokus pada tahap awal pembangunan compiler untuk Pascal-S, yaitu pembuatan *Lexical Analysis*. Di tahap ini, kami sudah membuat sebuah *lexical analyzer* (lexer) yang mampu membaca kode sumber Pascal-S dan memecahnya menjadi potongan-potongan bermakna yang disebut token, seperti *keyword*, *identifier*, *operator*, dan tanda baca lainnya. Proses ini kami lakukan dengan memanfaatkan konsep Deterministic Finite Automata (DFA), di mana setiap pola karakter diatur melalui transisi antar state. Kami juga harus membuat file aturan DFA serta diagram transisi yang menggambarkan bagaimana lexer dapat mengenali setiap token. Selain itu, kami juga sudah membuat diagram DFA yang digunakan, sehingga hubungan antara teori otomata yang kami pelajari dan implementasi program yang kami buat dapat terlihat lebih jelas.

Berdasarkan hasil pengujian terhadap beberapa file Pascal-S, lexer yang kami buat mampu mengidentifikasi dan mengklasifikasikan token dengan akurat, termasuk *keyword*, *identifier*, *operator* aritmatika dan relasional, serta literal seperti angka dan string. Program juga berhasil mengabaikan *whitespace* dan komentar. Setiap uji kasus menunjukkan bahwa hasil token yang keluar sesuai dengan format *output* yang diharapkan, menandakan bahwa proses analisis leksikal telah berjalan dengan benar.

4.2 Saran

Pada pembuatan kode program, dibutuhkan pemahaman tingkat lanjut terkait keseluruhan cara compiler dalam melakukan tahap Lexical Analyzer, Semantic dan Analysis. Karena hal ini merupakan masih pada tahap Lexical Analyzer, jadi asumsinya adalah ada beberapa bagian yang perlu disesuaikan sesuai tahapannya, seperti `NUMBER(-19)` untuk tahap awal yaitu Lexical Analyzer, namun untuk tahap selanjutnya seperti Semantic, maka seharusnya `OPERATOR(-) NUMBER(19)`

LAMPIRAN

A. Github

<https://github.com/DitaMaheswari05/PSC-Tubes-IF2224.git>

B. Workspace Diagram

https://drive.google.com/file/d/1NQ2A8T2x9A2pFk4XTADe3xN_BAh3VPpi/view?usp=sharing

C. Pembagian Tugas

NIM	Nama	Pekerjaan
13523125	Dita Maheswari	DFA rules , lexer (sedikit), laporan
13523127	Boye Mangaratua Ginting	Diagram, laporan
13523138	Samantha Laqueenna Ginting	Diagram, laporan
13523158	Lukas Raja Agripa	Lexer, laporan

REFERENSI

- Video Youtube, Introduction to Lexical Analyzer,  Introduction to Lexical Analyzer , diakses pada 17 Oktober 2025, pukul 13.52.